

```
struct sockaddr_can addr;
struct ifreq ifr;
strcpy(ifr.ifr_name, ifname);
ioctl(s, SIOCGIFINDEX, &ifr);
addr.can_family = AF_CAN;
addr.can_ifindex = ifr.ifr_ifindex;
bind(s, (struct sockaddr *)&addr, sizeof(addr));
```

**Step 3a - Writing CAN frames:** To send a frame with the ID 0x101 and a two byte payload of 0x41, 0x42, use the following commands:

```
struct can_frame frame;
frame.canid=0x101;
frame.can_dlc=2;

frame.data[0]=0x41; frame.data[1]=0x42;
int nbytes = write(s, &frame, sizeof(struct can_frame));
```

This can be called periodically on an even basis as per application needs.

**Step 3b - Reading CAN frames:** Use the following commands in this step:

```
nbytes = read(s, &frame, sizeof(struct can_frame));

//Extract frame.id, frame.can_dlc, frame.data[i]
```

This can be called repeatedly, preferably in an independent thread execution.

**Step 4 - Closing the socket:** Give the following command:

```
close(s);
```

## Language bindings

Wrappers are available in different languages to support CAN communication. A few bindings are listed below:

- Python comes with two types of implementations, through the package.
  - Native implementation with direct interfacing to kernel is available from Python 3.3 onwards.
  - The second implementation is based on *ctypes*, which is a wrapper around the standard C library; it is suitable for Python 2.7 and Python3 <3.4 (<https://python-can.readthedocs.io/en/2.1.0/interfaces/socketcan.html>).
- **Java:** This is a JNI wrapper by Entropia.de (<https://github.com/entropia/libsocket-can-java>).
- **Node.js:** This is through the npm package 'socketcan' (<https://github.com/sebi2k1/node-can>).
- Qt offers a good set of C++ APIs, signals and slot mechanisms for CAN communication via the serial bus module. These APIs are common for various back-ends

(<https://doc.qt.io/qt-5/qtcanbus-backends.html>).

- Apart from the SocketCAN back-end, Qt 5.12 has also added the VirtualCAN back-end which works over TCP. This is helpful for initial application development even on systems like Windows, which do not support the SocketCAN stack and APIs.
- The Android Automotive project offers HAL APIs and examples for CANBus integration.
- Wireshark is a popular packet analyser. It has a good plugin set for the dissection of raw CAN frames as well as analysing upper layer protocols like CANOpen, UDS and J1939.

Here is a list of a few libraries, toolkits and bridges built on top of SocketCAN:

- CANOpen stacks in Linux — CANFestival, CANOpenSocket
- Libraries and simulators for Unified Diagnostic Services (UDS), a vehicle diagnostic protocol (<https://github.com/openxc/uds-c> and <https://github.com/zombieCraig> (UDSim, uds-server, ICSim))
- A python package for the Vehicle Interface Toolkit
- Python package
- Carloop CAN Analyzer works with SocketCAN (Carloop.io)
- Kayak, a CANBus analysis tool
- Socketcand, a bridging interface between CANBus and TCP/IP networks
- Cannelloni, a tunnelling implementation of SocketCAN over Ethernet (<https://github.com/mguentner/cannelloni>)

The author hopes that the hints and links given will help the reader to explore the subject further. **END** 🐧

## Acknowledgement:

I am thankful to my team members Tracy Austina Z., A. Jegan and Namrata Mendis for helping me with CANBus concepts and reviewing the contents of this article.

## References

- [1] <https://www.kernel.org/doc/Documentation/networking/can.txt>
- [2] [https://www.can-cia.org/fileadmin/resources/documents/proceedings/2012\\_kleine-budde.pdf](https://www.can-cia.org/fileadmin/resources/documents/proceedings/2012_kleine-budde.pdf)
- [3] <https://github.com/linux-can/can-utils>
- [4] [https://www.elinux.org/CAN\\_Bus](https://www.elinux.org/CAN_Bus)
- [5] [http://processors.wiki.ti.com/index.php/AM335X\\_DCAN\\_Driver\\_Guide](http://processors.wiki.ti.com/index.php/AM335X_DCAN_Driver_Guide)

## By: Rajesh Sola

The author is an education specialist in the engineering academy at KPIT. He is involved in competency development in the automotive segment with a focus on infotainment systems.